

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XI
ERROR HANDLING



Disusun Oleh:

Neila Faaizah Asynur 105224028

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Praktikum modul 11 membahas tentang Error Handling. Fungsionalitas program berjalan sesuai alur normal merupakan prioritas, tetapi di dalam realitasnya program sering kali menghadapi situasi tidak terduga saat dieksekusi. Kejadian ini bisa terjadi karena kegagalan database, pembagian angka dengan nol, atau ada kesalahan pada inputan pengguna. Kejadian tidak terduga ini disebut dengan exception. Jika exception diabaikan dan tidak ditangani, sistem akan mengalami crash secara mendadak.

Java menyediakan mekanisme penanganan melalui fitur Exception Handling. Mekanisme ini penting karena mampu memisahkan antara logika bisnis utama dengan logika penanganan error. Dengan menggunakan implementasi try-catch-finally serta memanfaatkan keyword throw dan throws, alur kode akan lebih toleran terhadap kesalahan, memberikan pesan error yang informatif, serta memastikan resource program dapat dibersihkan dengan aman.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1.	kodeKereta	String	Menyimpan kode unik dari kereta
2.	namaKereta	String	Menyimpan nama dari setiap kereta
3.	rutePerjalanan	String	Menyimpan rute perjalanan kereta
4.	kursiKosong	int	Menyimpan jumlah kursi kosong yang tersedia
5.	nik	String	Meyimpan nik penumpang kereta
6.	namaPenumpang	String	Menyimpan nama penumpang kereta
7.	jumlahTiket	int	Menyimpan jumlah tiket yang akan dipesan oleh penumpang
8.	daftarKereta	List<KeretaApi>	Menyimpan daftar kereta api yang tersedia
9.	riwayatTransaksi	List<PesananTiket>	Menyimpan riwayat dari pemesanan tiket yang pernah terjadi
10.	kontroler	Reservasi	Objek yang berguna untuk menjalankan dan mengakses fitur-fitur dalam sistem pemesanan tiket.

11.	mulai	boolean	Kontrol utama perulangan untuk menjaga agar sistem dapat tetap berjalan atau berhenti secara dinamis
12.	pilihan	int	Menyimpan inputan pilihan dari pengguna
13.	kode	String	Menyimpan inputan kode kereta dari pengguna
14.	nik	String	Menyimpan inputan nik dari pengguna
15.	nama	String	Menyimpan inputan nama penumpang dari inputan pengguna
16.	jumlahTiket	int	Menyimpan inputan jumlah tiket yang ingin dibeli oleh pengguna

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1.	KeretaApi()	Constructor	Inisialisasi awal objek Kereta Api ketika pertama kali dibuat
2.	kurangiKursi (int jumlah)	Procedural	Mengurangi kursi yang tersedia dari kereta sesuai dengan tiket yang dibeli pengguna
3.	PesanTiket()	Constructor	Inisialisasi awal objek Pesan Tiket ketika pertama kali dibuat
4.	getNik()	Procedural	Mengakses value dari nik pengguna
5.	Reservasi()	Constructor	Inisialisasi awal objek Reservasi ketika pertama kali dibuat
6.	dataAwal()	Procedural	Menambahkan data awal kereta api yang telah tersedia di dalam sistem

7.	getDaftarKereta()	Procedural	Mengakses value yang tersimpan di dalam daftar kereta
8.	getRiwayatTransaksi()	Procedural	Mengakses value yang tersimpan di dalam riwayat transaksi
9.	pesanTiket()	Procedural	Melakukan pemrosesan transaksi pemesanan tiket kereta api sekaligus validasi aturan bisnis dalam melakukan transaksi
10.	main()	Procedural	Menjalankan dan mengeksekusi seluruh simulasi program

IV. Dokumentasi dan Pembahasan Code

Dokumentasi class KeretaApi

```

public class KeretaApi {
    public String kodeKereta;
    public String namaKereta;
    public String rutePerjalanan;
    public int kursiKosong;

    KeretaApi (String namaKereta, String kodeKereta, String rutePerjalanan, int kursiKosong){
        this.namaKereta = namaKereta;
        this.kodeKereta = kodeKereta;
        this.rutePerjalanan = rutePerjalanan;
        this.kursiKosong = kursiKosong;
    }

    public void kurangiKursi(int jumlah){
        this.kursiKosong -= jumlah;
    }
}

```

Gambar 1. Dokumentasi kode pada class KeretaApi

Dokumentasi class PesanTiket

```

public class PesanTiket {
    public String kodeKereta;
    private String nik;
    public String namaPenumpang;
    public int jumlahTiket;

    public PesanTiket(String kodeKereta, String nik, String namaPenumpang, int jumlahTiket){
        this.kodeKereta = kodeKereta;
        this.nik = nik;
        this.namaPenumpang = namaPenumpang;
        this.jumlahTiket = jumlahTiket;
    }

    public String getNik() {
        return nik;
    }
}

```

Gambar 2. Dokumentasi pada class PesanTiket

Dokumentasi class Reservasi

```
import java.util.*;

public class Reservasi {
    private List<KeretaApi> daftarKereta;
    private List<PesanTiket> riwayatTransaksi;

    public Reservasi(){
        daftarKereta = new ArrayList<>();
        riwayatTransaksi = new ArrayList<>();
        dataAwal();
    }

    private void dataAwal(){
        daftarKereta.add(new KeretaApi(namaKereta: "Argo Bromo", kodeKereta: "K01", rutePerjalanan: "JKT - SBY", kursiK... 50));
        daftarKereta.add(new KeretaApi(namaKereta: "Parahyangan", kodeKereta: "K02", rutePerjalanan: "JKT - BDG", kursiK... 15));
    }

    public List<KeretaApi> getDaftarKereta(){
        return daftarKereta;
    }

    public List<PesanTiket> getRiwayatTransaksi(){
        return riwayatTransaksi;
    }

    public void pesanTiket(String kodeKereta, String nik, String namaPenumpang, int jumlahTiket)throws RuteTidakDitemukanException, TiketHabisException{
        if (nik.length() != 16 || !nik.matches("[0-9]+")){
            throw new DataPenumpangTidakValidException(message: " >> error: NIK harus berisi tepat 16 angka!");
        }

        KeretaApi targetKereta = null;
        for (KeretaApi list : daftarKereta){
            if (list.kodeKereta.equalsIgnoreCase(kodeKereta)){
                targetKereta = list;
                break;
            }
        }

        if (targetKereta == null){
            throw new RuteTidakDitemukanException(message: " >> error: Rute perjalanan tidak ditemukan!");
        }

        if (jumlahTiket > targetKereta.kursiKosong){
            throw new TiketHabisException(message: " >> error: Maaf kursi kosong tidak tersedia", targetKereta.namaKereta, targetKereta.kursiKosong);
        }

        targetKereta.kurangKursi(jumlahTiket);
        riwayatTransaksi.add(new PesanTiket(targetKereta.kodeKereta, nik, namaPenumpang, jumlahTiket));
    }
}
```

Gambar 3. Dokumentasi pada class Reservasi

Dokumentasi class DataPenumpangTidakValidException

```
public class DataPenumpangTidakValidException extends RuntimeException{
    public DataPenumpangTidakValidException (String message){
        super(message);
    }
}
```

Gambar 4. Dokumentasi pada class DataPenumpangTidakValidException

Dokumentasi class RuteTidakDitemukanException

```
public class RuteTidakDitemukanException extends Exception{
    public RuteTidakDitemukanException (String message){
        super(message);
    }
}
```

Gambar 5. Dokumentasi pada class RuteTidakDitemukanException

Dokumentasi class TiketHabisException

```
public class TiketHabisException extends Exception{
    public String namaKereta;
    public int kursiKosong;

    public TiketHabisException (String message, String namaKereta, int kursiKosong){
        super(message);
        this.namaKereta = namaKereta;
        this.kursiKosong = kursiKosong;
    }
}
```

Gambar 6. Dokumentasi pada class TiketHabisException

Dokumentasi main program (JavaExpress)

```
import java.util.*;

public class JavaExpress {
    Run | Debug
    public static void main(String[] args) throws Exception {
        Scanner input = new Scanner(System.in);

        Reservasi kontroler = new Reservasi();
        boolean mulai = true;

        while (mulai){
            System.out.println(" === MENU UTAMA ===");
            System.out.println("1. Lihat jadwal kereta");
            System.out.println("2. Pesan tiket kereta");
            System.out.println("3. Keluar");
            System.out.print("Pilih menu (1-3): ");

            try {
                int pilihan = input.nextInt();
                input.nextLine();

                switch (pilihan) {
                    case 1:
                        for (KeretaApi list : kontroler.getDaftarKereta()){
                            System.out.println("\n=====");
                            System.out.println("Kode Kereta: " + list.kodeKereta);
                            System.out.println("Nama Kereta: " + list.namaKereta);
                            System.out.println("Rute Perjalanan: " + list.rutePerjalanan);
                            System.out.println("Kursi Kosong Tersedia: " + list.kursiKosong + " kursi");
                            System.out.println("=====");
                        }
                        break;

                    case 2:
                        System.out.println("Input kode kereta: ");
                        String kode = input.nextLine().trim();

```

```

        System.out.println("Input NIK: ");
        String nik = input.nextLine().trim();

        System.out.println("Nama penumpang: ");
        String nama = input.nextLine().trim();

        System.out.println("Jumlah Tiket: ");
        int jumlahTiket = input.nextInt();
        input.nextLine();

        kontroler.pesanTiket(kode, nik, nama, jumlahTiket);
        break;

    case 3:
        mulai = false;
        break;
    default:
        System.out.println("Pilihan tidak valid!");
        break;
    }
} catch (InputMismatchException e) {
    System.out.println(">> error: inputan salah! Masukkan angka yang valid!");
} catch (DataPenumpangTidakValidException e){
    System.out.println(e.getMessage());
} catch (RuteTidakDitemukanException e){
    System.out.println(e.getMessage());
} catch (TiketHabisException e){
    System.out.println(e.getMessage());
} finally{
    if (!mulai){
        System.out.println("Terimakasih telah menggunakan sistem Java Express!");
        input.close();
    }
}
}
}
}

```

Gambar 7. Dokumentasi pada main program

Untuk memahami alur kerja dari kode ini, berikut dijelaskan pembahasan kode dimulai dari kode pada class KeretaApi:

```

public String kodeKereta;
public String namaKereta;
public String rutePerjalanan;
public int kursiKosong;

```

Pada kode bagian ini, berguna untuk mendefinisikan atribut apa saja yang dibutuhkan dalam class KeretaApi ini.

```

KeretaApi (String namaKereta, String kodeKereta, String rutePerjalanan, int
kursiKosong){
    this.namaKereta = namaKereta;
    this.kodeKereta = kodeKereta;
    this.rutePerjalanan = rutePerjalanan;
    this.kursiKosong = kursiKosong;
}

```

Blok ini merupakan constructor dari class KeretaApi. Constructor ini berguna untuk membuat objek awal pada pendefinisian objek.

```

public void kurangiKursi(int jumlah){
    this.kursiKosong -= jumlah;
}

```

```
}
```

Di dalam class KeretaApi, ada sebuah method kurangiKursi() yang berguna untuk mengurangi total ketersediaan kursi yang ada di dalam kereta api. Hal ini berguna untuk menghindari pemesanan pada kursi yang tidak tersedia.

Setelah itu, masuk ke class PesanTiket:

```
public String kodeKereta;  
private String nik;  
public String namaPenumpang;  
public int jumlahTiket;
```

Pada bagian kode ini berguna untuk mendefinisikan sebuah keseluruhan atribut yang akan digunakan di dalam class PesanTiket.

```
public PesanTiket(String kodeKereta, String nik, String namaPenumpang, int  
jumlahTiket){  
    this.kodeKereta = kodeKereta;  
    this.nik = nik;  
    this.namaPenumpang = namaPenumpang;  
    this.jumlahTiket = jumlahTiket;  
}
```

Blok ini merupakan constructor yang berguna untuk menginisialisasikan objek diawal pembuatan.

```
public String getNik() {  
    return nik;  
}
```

Blok kode ini merupakan getter yang digunakan untuk mengakses value dari atribut nik yang terenkapsulasi di class PesanTiket.

Setelah selesai di class PesanTiket, lanjutkan ke class Reservasi

```
private List<KeretaApi> daftarKereta;  
private List<PesanTiket> riwayatTransaksi;
```

Pada class Reservasi, ada penggunaan collection List yang berguna untuk menyimpan data daftar kereta api yang tersedia di sistem, dan untuk menyimpan riwayat transaksi yang pernah dilakukan di dalam sistem.

```
public Reservasi(){  
    daftarKereta = new ArrayList<>();  
    riwayatTransaksi = new ArrayList<>();  
    dataAwal();  
}
```

Kode ini merupakan constructor dari Reservasi yang berguna untuk mengaktifkan collecection yang diinisialisasi sebelumnya dan masuk ke dalam method dataAwal().

```
private void dataAwal(){
    daftarKereta.add(new KeretaApi("Argo Bromo", "K01", "JKT - SBY", 50));
    daftarKereta.add(new KeretaApi("Parahyangan", "K02", "JKT - BDG", 15));
}
```

Method ini berguna untuk melakukan penginisialisasian kereta yang tersedia di dalam sistem.

```
public List<KeretaApi> getDaftarKereta(){
    return daftarKereta;
}
public List<PesanTiket> getRiwayatTransaksi(){
    return riwayatTransaksi;
}
```

Blok kode ini merupakan getter yang berguna untuk mengakses value yang berada di dalam daftar kereta dan riwayat transaksi yang terenkapsulasi di class Reservasi.

```
public void pesanTiket(String kodeKereta, String nik, String namaPenumpang, int
jumlahTiket)throws RuteTidakDitemukanException, TiketHabisException{
    if (nik.length() != 16 || !nik.matches("[0-9]+")){
        throw new DataPenumpangTidakValidException(" >> error: NIK harus
berisi tepat 16 angka!");
    }
```

```
        KeretaApi targetKereta = null;
        for (KeretaApi list : daftarKereta){
            if (list.kodeKereta.equalsIgnoreCase(kodeKereta)){
                targetKereta = list;
                break;
            }
        }

        if (targetKereta == null){
            throw new RuteTidakDitemukanException(" >> error: Rute perjalanan tidak
ditemukan!");
        }
```

```
        if (jumlahTiket > targetKereta.kursiKosong){
            throw new TiketHabisException(" >> error: Maaf kursi kosong tidak
tersedia!", targetKereta.namaKereta, targetKereta.kursiKosong);
        }
```

```
        targetKereta.kurangiKursi(jumlahTiket);
        riwayatTransaksi.add(new PesanTiket(targetKereta.kodeKereta, nik,
namaPenumpang, jumlahTiket));
```

```
}
```

Blok kode ini merupakan sebuah method yang berguna untuk melakukan validasi terhadap pemesanan tiket yang dilakukan oleh pengguna. Ada 3 jenis custom exception yang dibuat dengan tujuan yang disesuaikan. Jika nik tidak terdiri dari 16 angka, atau nik diisi dengan karakter lain selain angka, maka program akan memanggil `DataPenumpangTidakValidException` untuk dimunculkan di layar pengguna. Hal ini akan memberitahukan kepada pengguna apa error yang terjadi secara khusus.

Jika nik pengguna telah sesuai, lanjut ke bagian pengguna melakukan penginputan kode kereta yang ingin dipesan. Jika kode kereta api yang diinputkan pengguna tidak tersedia di dalam sistem, maka sistem akan memberikan message kepada pengguna bahwa kode tersebut tidak valid.

Jika nik dan kode kereta api telah benar, kode akan melakukan validasi pada total kursi kosong yang tersedia dengan total tiket yang akan dibeli oleh pengguna. Jika kursi tidak mencukupi, maka sistem akan memberikan message kepada pengguna jika total kursi yang tersedia tidak mencukupi untuk total pembelian tiket yang ingin dilakukan.

Lanjutkan dengan memahami kode yang berada di main program:

```
Scanner input = new Scanner(System.in);
```

```
Reservasi kontroler = new Reservasi();
```

```
boolean mulai = true;
```

Pada blok ini adalah menginisialisasi objek `Reservasi` untuk dapat menggunakan fitur-fitur dalam melakukan simulasi pemesanan tiket di dalam sistem `JavaExpress`. Penggunaan `Scanner` di awal merupakan pembuatan objek dengan tipe `Scanner` sebagai objek penerima inputan dari pengguna. Dan variabel `mulai` berguna untuk pengendali loop agar dapat berjalan dan berhenti secara dinamis sesuai dengan inputan pengguna.

```
while (mulai){  
    System.out.println(" === MENU UTAMA ===");  
    System.out.println("1. Lihat jadwal kereta");  
    System.out.println("2. Pesan tiket kereta");  
    System.out.println("3. Keluar");  
    System.out.print("Pilih menu (1-3): ");
```

Bagian ini merupakan bagian awal dari loop yang akan menampilkan menu yang tersedia di dalam sistem `Java Express`. Pengguna dapat melakukan pemesanan tiket atau sekedar melihat jadwal dari kereta di dalam sistem ini. Selain 2 pilihan tersebut, pengguna dapat keluar dari sistem dan menghentikan sistem ini.

```
try {  
    int pilihan = input.nextInt();  
    input.nextLine();  
  
    switch (pilihan) {
```

```

case 1:
    for (KeretaApi list : kontroler.getDaftarKereta()){
        System.out.println("\n=====");
        System.out.println("Kode Kereta: " + list.kodeKereta);
        System.out.println("Nama Kereta: " + list.namaKereta);
        System.out.println("Rute Perjalanan: " + list.rutePerjalanan);
        System.out.println("Kursi Kosong Tersedia: " + list.kursiKosong
+ " kursi");
        System.out.println("=====");
    }
    break;

case 2:
    System.out.println("Input kode kereta: ");
    String kode = input.nextLine().trim();

    System.out.println("Input NIK: ");
    String nik = input.nextLine().trim();

    System.out.println("Nama penumpang: ");
    String nama = input.nextLine().trim();

    System.out.println("Jumlah Tiket: ");
    int jumlahTiket = input.nextInt();
    input.nextLine();

    kontroler.pesanTiket(kode, nik, nama, jumlahTiket);
    break;

case 3:
    mulai = false;
    break;
default:
    System.out.println("Pilihan tidak valid!");
    break;
}
}

```

Pada blok kode ini, merupakan sebuah percabangan yang akan dijalankan sesuai dengan inputan pengguna di awal. Jika pengguna memilih untuk melihat jadwal dari kereta api yang tersedia di dalam sistem, maka sistem akan memperlihatkan seluruh detail dari kereta api ke layar pengguna.

Jika pengguna memilih menu kedua atau pemesanan tiket, akan dibutuhkan inputan pengguna untuk dapat melanjutkan ke tahap selanjutnya. Sistem akan meminta inputan berupa kode kereta, nik, nama penumpang, serta jumlah tiket yang ingin dipesan. Setelah keseluruhan ini diinput oleh pengguna, sistem akan

mengirimkan data tersebut untuk memanggil method pesanTiket. Di dalam method tersebut, akan dilakukan validasi yang diperlukan.

Jika pengguna memilih untuk menu ketiga atau keluar dari sistem, value variabel mulai akan diubah menjadi false dan perulangan akan berhenti dilakukan.

Jika pilihan pengguna tidak ada di antara menu yang ada, maka sistem akan memberikan message kepada pengguna jika pilihan yang dipilih tidak valid.

```
catch (InputMismatchException e) {
    System.out.println(" >> error: inputan salah! Masukkan angka yang
valid!");
    } catch (DataPenumpangTidakValidException e){
        System.out.println(e.getMessage());
    } catch (RuteTidakDitemukanException e){
        System.out.println(e.getMessage());
    } catch (TiketHabisException e){
        System.out.println(e.getMessage());
    } finally{
        if (!mulai){
            System.out.println("Terimakasih telah menggunakan sistem Java
Express!");
            input.close();
        }
    }
}
```

Blok catch inilah yang akan berguna untuk menangkap error yang terjadi dan mengembalikan ke layar pengguna berupa pesan yang informatif. Tanpa adanya blok catch ini, ketika ada kesalahan inputan dari pengguna maka sistem tidak akan melakukan apapun dan tetap menjalankan kode jika hal itu memungkinkan. Tetapi seringkali kesalahan tidak terduga dari inputan pengguna dapat menyebabkan program menjadi crash mendadak.

Blok finally ini akan selalu dijalankan, sehingga ketika keseluruhan kode telah dilakukan akan menampilkan message yang ada di dalam blok ini dan kemudian objek Scanner akan ditutup agar tidak memakan memori.

V. Kesimpulan

Kesimpulan dari praktikum di modul 11 ini adalah penerapan mekanisme Exception Handling melalui try-catch-finally terbukti efektif dalam menjaga stabilitas dari sebuah program. Program tidak mengalami crash secara mendadak ketika menghadapi kejadian tidak terduga saat runtime, terutama ketika adanya penginputan huruf pada menu yang membutuhkan data numerik. Penggunaan Checked dan Unchecked Exception mampu memberikan informasi yang jelas terkait kesalahan yang terjadi, dan pemanfaatan blok finally memberikan jaminan dalam resource kompute karena akan memastikan penutupan objek Scanner dan mencetak message penutup sistem.

VI. Daftar Pustaka

Tim Asisten Laboratorium Komputer. (2026). Modul Praktikum Pemrograman Berorientasi Objek - Modul 11: Error Handling. Program Studi Ilmu Komputer, Universitas Pertamina